物联网多传感器数据采集与无线传输系统设计

摘要

本项目面向“物联网系统设计作业”的要求，完成了一个具备前后端交互的轻量级 IoT 监测系统：后端以 STM32F103 为核心，采集 DHT11 温湿度与光敏传感器（ADC）数据，经串口与 ESP8266（AT 指令）通信，通过 Wi-Fi 将数据以 HTTP 请求上传至 PC 端的 Node-RED 应用平台；前端由 Node-RED 的调试与可视化组件实现对数据的实时展示与验证。系统在实现过程中重点解决了开发板 USART1 与 ESP8266 占用冲突、AT 指令链路调通、网络环境（校园网认证）不可直连导致的连接失败等问题，最终通过手机热点方式完成稳定上报，并在 Node-RED Debug 面板中成功解析为结构化字段（temp/humi/light）。本报告给出系统方案论证、硬件连接、软件实现、通信协议、测试流程与问题复盘，形成可复用的工程工作流。

关键词：STM32F103；DHT11；ADC 光敏；ESP8266；AT 指令；HTTP；Node-RED；物联网数据上报。

1. 引言

1.1 课题背景与目标

物联网系统的核心链路通常由“感知层—网络层—平台层—应用层”构成。本作业要求完成一个能够体现数据采集、传输、存储/处理与前端展示的系统，并提交设计报告、源码与演示材料。基于现有硬件条件（STM32F103 开发板 + DHT11 + 光敏传感器 + ESP8266 模块）与软件能力（Keil5、标准外设库、Node-RED、基础网络调试），本项目将目标定义为：

采集端（后端）：STM32F103 周期性读取温湿度与光照强度（百分比或归一化值）。

联网端（网关）：STM32 通过 UART 驱动 ESP8266（AT 固件），完成 Wi-Fi 入网与 TCP/HTTP 数据发送。

平台端（前端/应用平台）：PC 上运行 Node-RED，提供 HTTP 接口接收数据，并在调试窗口/仪表盘展示，实现“采集—上传—解析—可视化”的闭环。

1.2 总体思路与可行性分析

STM32F103 对 DHT11 采用单总线时序，光敏模块通过 ADC 读取，均属于典型嵌入式采集任务，成熟可靠。

ESP8266 AT 模式可直接通过 UART 控制入网与发送数据，避免在 STM32 上移植复杂 TCP/IP 协议栈。

Node-RED 提供零代码/低代码的数据接入、解析与可视化能力，适合课程作业快速落地与演示。

2. 系统总体方案设计

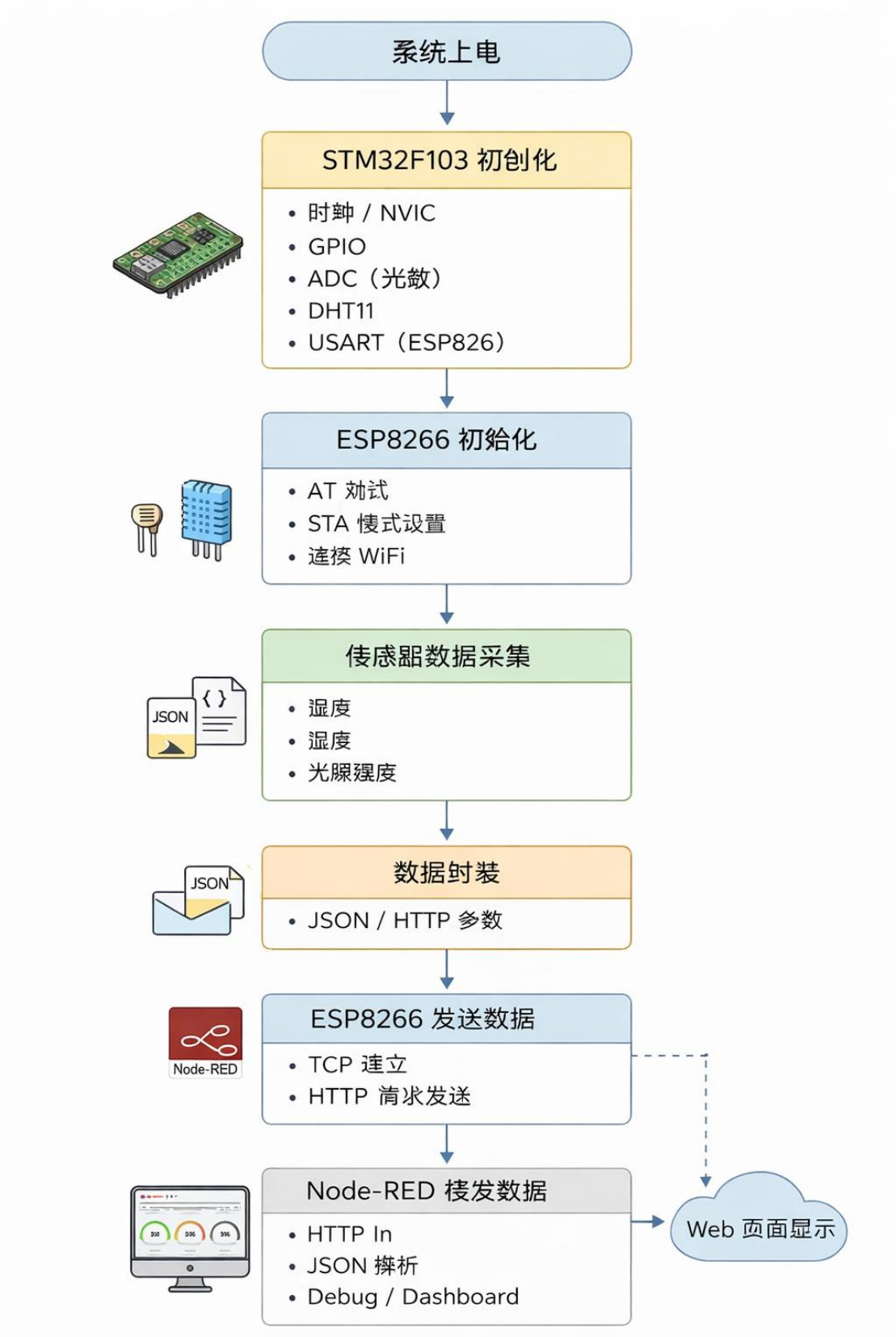
2.1 系统架构

系统采用典型“单片机采集 + Wi-Fi 模块上传 + PC 平台接收”的结构：感知层：DHT11（温湿度），光敏传感器（模拟量→ADC）。

控制与边缘处理：STM32F103 完成采集、滤波（可选）、数据格式化（JSON/URL 参数）、发送调度。

网络层：ESP8266（AT）连接 Wi-Fi，建立到 PC 的 TCP 连接并发送 HTTP 请求。

平台与应用层：Node-RED 提供 HTTP In 节点接收数据，JSON 节点解析，Debug / Dashboard 展示。


系统上电
STM32F103 初创化
• 时蚺 / NVIC
• GPIO
• ADC（光敛）
• DHT11
• USART（ESP826）
ESP8266 初姶化
• AT 劝式
• STA 悽式设畺
• 连揆 WiFi
伎感郢数据采集
• 湜度
• 湜度
• 光腺猩度
JSON
数据时装
• JSON / HTTP 多数
JSON
ESP8266 发送数据
• TCP 连立
• HTTP 淆氷发送
Node-RED
Node-RED 桋发数据
• HTTP In
• JSON 擀析
• Debug / Dashboard
Web 页面显示

2.2 数据流与交互流程

数据链路（一次上报）可抽象为：

STM32 采集温度 temp、湿度 humi、光照 light。

STM32 通过 UART 向 ESP8266 发送 AT 指令：

入网：AT+CWJAP="SSID","PWD"

建连：AT+CIPSTART="TCP","PC_IP",1880

发送：AT+CIPSEND=len + HTTP 请求内容

Node-RED 在 /sensor 接口收到数据，解析为结构化字段，在 Debug 中显示并可进一步写入数据库或仪表盘组件。

2.3 协议选择说明（HTTP vs MQTT）

本项目选择 HTTP 的原因：

Node-RED 原生 HTTP In/Response 节点配置简单，便于验收演示；

AT 指令走 HTTP 请求链路直观，抓包/调试更易定位问题；

3. 系统硬件设计

3.1 硬件清单（BOM 表）

STM32F103 开发板（正点原子精英版或同类）

DHT11 温湿度模块

光敏传感器模块（输出模拟量至 ADC）

ESP8266 AT 模块（如 ATK-ESP8266-V1.3）

杜邦线若干、电源线

供电：开发板 5V/USB；ESP8266 需 3.3V 稳定供电（峰值电流较大）

3.2 传感器接口设计

3.2.1 DHT11 接口

单总线数据线连接到 STM32 GPIOG_PIN_11

时序要求严格，需要微秒级延时函数 delay_us()。

输出数据：整数温度（0–50℃）与湿度（20–98%RH）

DHT11 引脚	连接到 STM32
VCC	3.3V 或 5V（常用 3.3V）
DATA	任意 GPIO（如 PA1 / PB8 等）
GND	GND

DATA 引脚必须上拉（模块一般自带 5.1kΩ / 10kΩ 上拉电阻）

DATA 线不是 ADC，不是串口，而是普通 GPIO。

DHT11 通过单总线数字通信与 STM32 连接，使用 GPIO 口在输出模式下发送起始信号，在输入模式下接收传感器返回的脉冲序列。通过对高电平持续时间的精确测量，解码出温湿度数据。

3.2.2 光敏传感器（ADC）接口

光敏模块模拟输出接 STM32 ADC3 通道，PF8 。

软件侧通过多次采样平均降低噪声，并映射到 0–100% 光照强度。

输出数据：Light = 0~100（归一化百分比）。

LDR 模块	STM32
VCC	3.3V
GND	GND
AO	ADC 3 GPIOF_PIN_8

STM32 中 ADC 的配置要点：

ADC 模式：单次转换

数据对齐：右对齐

分辨率：12-bit（0–4095）

采样时间：较长（如 239.5 cycles，稳定）

软件层面的“光强百分比”是怎么算的：light_val = Lsens_Get_Val();

内部逻辑通常是：

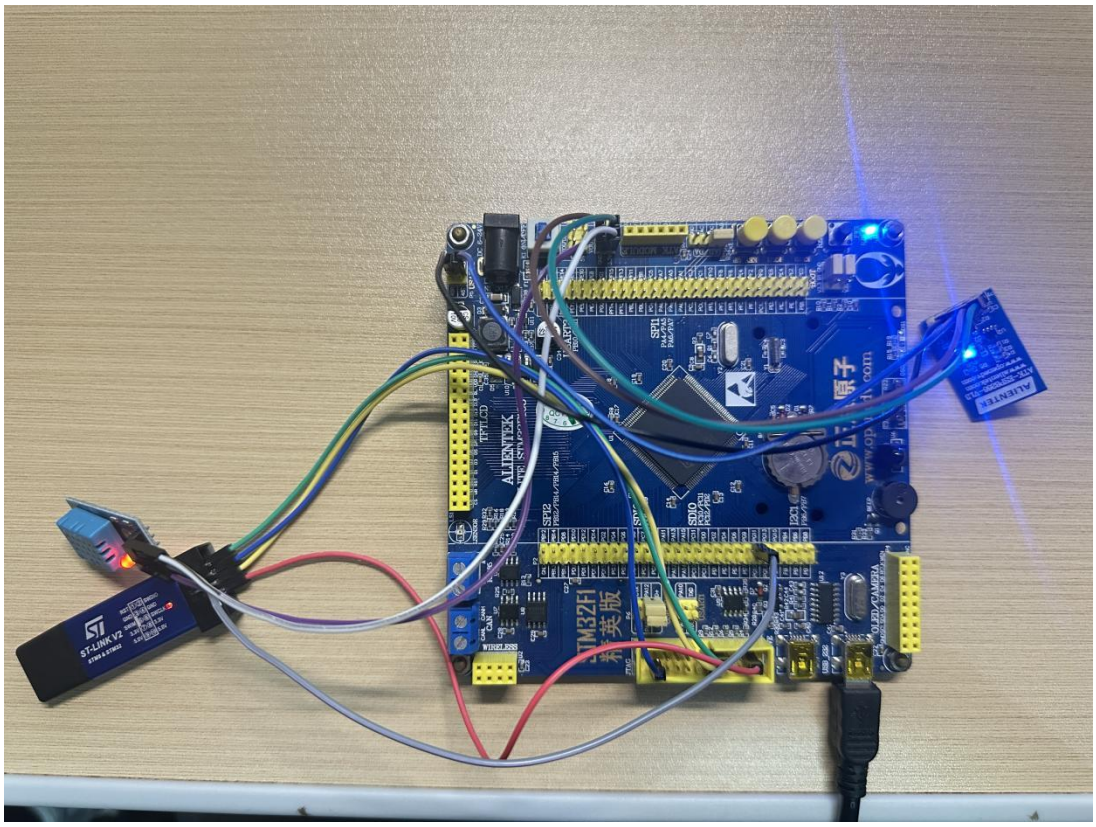
读取 ADC 原始值（0–4095）

映射为百分比

light = (4095 - adc_value) * 100 / 4095;

光越强 → LDR 电阻越小 → ADC 电压变化 → 数值变化

光敏电阻模块通过电阻分压将光照强度转换为模拟电压信号，并由 STM32 的 ADC 模块进行采样和量化，实现对环境光强的实时测量。



3.3 ESP8266 与串口资源冲突说明

使用的 F103 开发板上，USART1 的 PA9/PA10 通常被板载 USB-串口（或外设接口）占用用于 PC 串口调试。当把 ESP8266 接到 PA9/PA10 后，会出现两类典型问题：

硬件冲突：USART1 同时连着“板载 USB-串口芯片/接口”和 ESP8266，导致电平驱动冲突、串口乱码或完全无输出。

软件输出重定向失效：工程 fputc() 是写 USART1->DR，当 USART1 通道被占用/异常时，printf() 无法正常显示。

因此工程上必须做“串口分工。

USART1 留给 PC 串口调试（或放弃 PC 调试，完全给 ESP8266 用）

另一路 USART（如 USART2）给 ESP8266（推荐）

最终能跑通上报链路，核心也是完成了“ESP8266 使用独立 UART +

Node-RED 接收”的闭环。

4. 系统软件设计

本项目软件分为三层：STM32 采集层、ESP8266 AT 控制层、Node-RED 平台层。

```c
#include "sys.h"
#include "delay.h"
#include "led.h"
#include "usart.h"
#include "dht11.h"
#include "lsens.h"
#include <stdio.h>
#include <string.h>   // ★ 必须加

/* ================= ESP8266 发送 ================= */
void ESP8266_SendString(char *str)
{
    while (*str)
    {
        USART_SendData(USART2, *str++);
        while (USART_GetFlagStatus(USART2, USART_FLAG_TXE) == RESET);
    }
}

int main(void)
{
    /* ===== 所有变量必须放在最前面（C89） ===== */
    u8 temp = 0;
    u8 humi = 0;
    u8 light = 0;

    char json[128];
    char http[512];
    char cmd[64];

    delay_init();
    LED_Init();
    uart2_init(115200);   // USART2 -> ESP8266
    Lsens_Init();

    /* DHT11 初始化 */
    while (DHT11_Init())
    {
```

```c
    while (DHT11_Init())
    {
        LED0 = !LED0;
        delay_ms(300);
    }

    delay_ms(3000);   // 等 ESP8266 上电

    /* ===== ESP8266 基本配置 ===== */
    ESP8266_SendString("AT\r\n");
    delay_ms(1000);

    ESP8266_SendString("AT+CWMODE=1\r\n");
    delay_ms(1000);

    ESP8266_SendString(
        "AT+CWJAP=\"Archi\",\"88888888\"\r\n"
    );
    delay_ms(6000);

    LED0 = 1;   // WiFi OK

    while (1)
    {
        /* 采集传感器 */
        DHT11_Read_Data(&temp, &humi);
        light = Lsens_Get_Val();

        /* 建立 TCP 连接 */
        ESP8266_SendString(
            "AT+CIPSTART=\"TCP\",\"172.20.10.2\",1880\r\n"
        );
        delay_ms(2000);

        /* JSON 数据 */
        sprintf(json,
            "{\"temp\":%d,\"humi\":%d,\"light\":%d}",
            temp, humi, light
```

```c
        DHT11_Read_Data(&temp, &humi);
        light = Lsens_Get_Val();

        /* 建立 TCP 连接 */
        ESP8266_SendString(
            "AT+CIPSTART=\"TCP\",\"172.20.10.2\",1880\r\n"
        );
        delay_ms(2000);

        /* JSON 数据 */
        sprintf(json,
            "{\"temp\":%d,\"humi\":%d,\"light\":%d}",
            temp, humi, light
        );

        /* HTTP POST */
        sprintf(http,
            "POST /sensor HTTP/1.1\r\n"
            "Host: 172.20.10.2\r\n"
            "Content-Type: application/json\r\n"
            "Content-Length: %d\r\n\r\n"
            "%s",
            strlen(json), json
        );

        /* 通知发送长度 */
        sprintf(cmd, "AT+CIPSEND=%d\r\n", strlen(http));
        ESP8266_SendString(cmd);
        delay_ms(1000);

        /* 发送数据 */
        ESP8266_SendString(http);

        LED0 = !LED0;
        delay_ms(200);
    }
}
```

主函数设计逻辑

#include "sys.h"

系统底层配置头文件

包含 STM32 的时钟、NVIC、中断等基础定义

是正点原子工程的核心基础文件之一

#include "delay.h"

延时函数库

提供 delay_ms()、delay_us() 等

在 ESP8266 启动等待、AT 指令间隔中大量使用

#include "led.h"

板载 LED 驱动

使用 LED0 来指示系统状态（初始化、WiFi 成功、发送成功）

#include "usart.h"

串口驱动

本工程中：

USART2 → 专门用于与 ESP8266 通信

uart2_init(115200) 在这里声明

#include "dht11.h"

DHT11 温湿度传感器驱动

提供:

DHT11_Init()

DHT11_Read_Data(&temp, &humi)

#include "lsens.h"

光敏传感器驱动

底层使用 ADC

提供 Lsens_Get_Val() 返回光照百分比

#include <stdio.h>

标准 C 库

提供 sprintf() 等字符串格式化函数

#include <string.h> // ★ 必须加字符串处理库

提供 strlen()

HTTP 协议里必须计算 Content-Length，所以必不可少

ESP8266 串口发送函数

void ESP8266_SendString(char *str)

向 ESP8266 发送字符串

通过 USART2（PA2 / PA3）

while (*str)

遍历字符串，直到 \0 结束

USART_SendData(USART2, *str++);

每次发送一个字符

str++ 表示发送完当前字符后指针后移

while (USART_GetFlagStatus(USART2, USART_FLAG_TXE) == RESET);

等待串口发送缓冲区空

保证每个字符都发送完成

否则容易丢字符（ESP8266 对 AT 指令很敏感）

这一函数的作用总结一句话：

STM32 → ESP8266 的“发话筒”

main 函数整体结构

int main(void)

程序入口

变量定义（必须在最前面，C89 规范）

u8 temp = 0;

u8 humi = 0;

u8 light = 0;

存储传感器采集值

使用 u8（0–255）完全够用

DHT11 和光敏都不会超范围

char json[128];

用来存放 JSON 格式的传感器数据

示例:

{"temp":26,"humi":50,"light":22}

char http[512];

用来存放 完整 HTTP 请求

包含:

请求头

Content-Length

JSON 数据体

char cmd[64];

存放 ESP8266 的 AT 指令

如：AT+CIPSEND=123

为什么要放在 main 最前面?

因为 Keil 默认是 C89 标准，不允许“先执行代码再定义变量”。

STM32 基础初始化

delay_init();

初始化 SysTick 定时器

为后面的 delay_ms() 提供支持

LED_Init();

初始化板载 LED GPIO

uart2_init(115200);

初始化 USART2

波特率 115200

这是 STM32 ↔ ESP8266 的通信通道

Lsens_Init();

初始化光敏传感器

实际上是 ADC 初始化

DHT11 初始化流程

while (DHT11_Init())

如果初始化失败（返回非 0）

说明 DHT11 没接好或没响应

LED0 = !LED0;

delay_ms(300);

LED 快闪作为 错误指示

直到 DHT11 初始化成功才会跳出

等待 ESP8266 上电完成

delay_ms(3000);

ESP8266 上电需要时间

不等的话 AT 指令会无响应

这是非常关键的一步

ESP8266 初始化（AT 指令）

ESP8266_SendString("AT\r\n");

测试 ESP8266 是否在线

等价于“你在串口里手动敲 AT”

ESP8266_SendString("AT+CWMODE=1\r\n");

设置为 STA 模式

即：连接路由器 / 热点

ESP8266_SendString(

```
        "AT+CWJAP=\"Archi\",\"88888888\"\r\n"
);
```

连接 WiFi

"Archi"：热点名称

"88888888"：热点密码

必须和你电脑开的热点完全一致

```
delay_ms(6000);
```

等待 WiFi 连接完成

校园网 / 热点通常需要 3–6 秒

```
LED0 = 1;
```

LED 常亮

表示 WiFi 已连接成功

主循环：采集 + 上传

```
while (1)
```

无限循环

实现 周期性采集 + 上传

采集传感器数据

```
DHT11_Read_Data(&temp, &humi);
```

读取温度和湿度

通过指针返回数据

```
light = Lsens_Get_Val();
```

读取光敏 ADC 值

返回 0–100 的百分比

建立 TCP 连接（Node-RED）

```
ESP8266_SendString(
    "AT+CIPSTART=\"TCP\",\"172.20.10.2\",1880\r\n"
);
```

连接 Node-RED 服务器

172.20.10.2：你电脑的 IPv4

1880：Node-RED 默认端口

```
delay_ms(2000);
```

等待 TCP 建立完成

构造 JSON 数据

```
sprintf(json,
    "{\"temp\":%d,\"humi\":%d,\"light\":%d}",
    temp, humi, light
);
```

把传感器数据封装成 JSON

与 Node-RED 中 json 节点完全匹配

构造 HTTP POST 请求

```
sprintf(http,
        "POST /sensor HTTP/1.1\r\n"
        "Host: 172.20.10.2\r\n"
        "Content-Type: application/json\r\n"
        "Content-Length: %d\r\n\r\n"
        "%s",
        strlen(json), json
);
```

这是整个项目最核心的一段：

请求路径：/sensor

对应 Node-RED 的 http in 节点

Content-Length 必须严格正确

空行 \r\n\r\n 后才是 JSON 数据

通知 ESP8266 要发送多少字节

```
sprintf(cmd, "AT+CIPSEND=%d\r\n", strlen(http));
```

ESP8266 必须提前知道发送长度

否则不会进入发送状态

ESP8266_SendString(cmd);

delay_ms(1000);

等待 ESP8266 返回 > 提示符

真正发送 HTTP 数据

ESP8266_SendString(http);

数据通过 WiFi 发给 Node-RED

Node-RED 立即解析并显示

LED0 = !LED0;

每成功发送一次 LED 翻转

作为运行状态指示

delay_ms(200);

控制上传周期

防止过快刷屏或网络拥塞

4.1 STM32 端总体软件结构

初始化：时钟/延时、GPIO、ADC、DHT11、串口（给 ESP8266）。

ESP8266 初始化：等待上电稳定 → 发送 AT 测试 → 设置 STA 模式 → 连接热点。

周期采集：每 0.2 秒采集一次传感器。

周期上报：组包 JSON 或 URL 参数，发送 HTTP 请求到 Node-RED。

异常处理：若建连失败，关闭连接重试；若 Wi-Fi 断开，重连 AP。

4.2 ESP8266 AT 指令工作原理与关键指令

在 AT 固件模式下，ESP8266 本质是“串口控制的网络协处理器”。常用命令：

连通性：AT（期望返回 OK）

模式：AT+CWMODE=1（STA 模式）

连接热点：AT+CWJAP="SSID","PASSWORD"

查询 IP：AT+CIFSR

建 TCP：AT+CIPSTART="TCP","ip",port

指定发送长度：AT+CIPSEND=len

关闭连接：AT+CIPCLOSE

4.3 Node-RED 平台搭建与流程配置

4.3.1 Node-RED 运行与访问

Node-RED 默认编辑器：http://localhost:1880

Dashboard 默认页面：http://localhost:1880/ui

成功看到 Dashboard 欢迎页与编辑器画布，说明平台侧正常。

4.3.2 Flow 设计（HTTP 接收 + JSON 解析 + Debug）

当前流程的核心节点是：

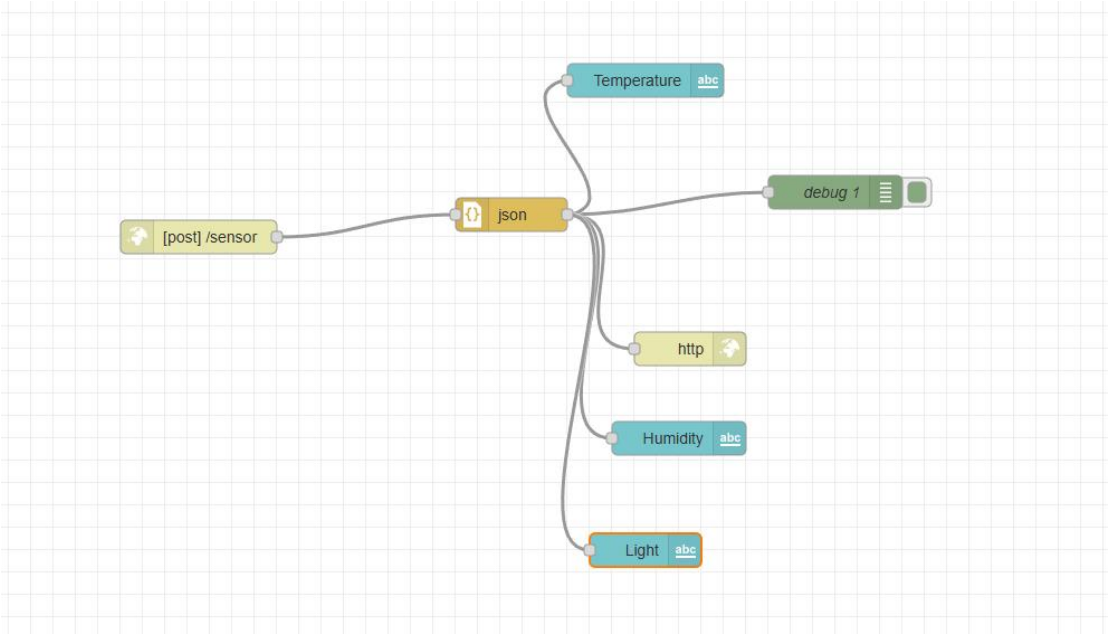
HTTP In：方法 POST（或 GET），路径 /sensor

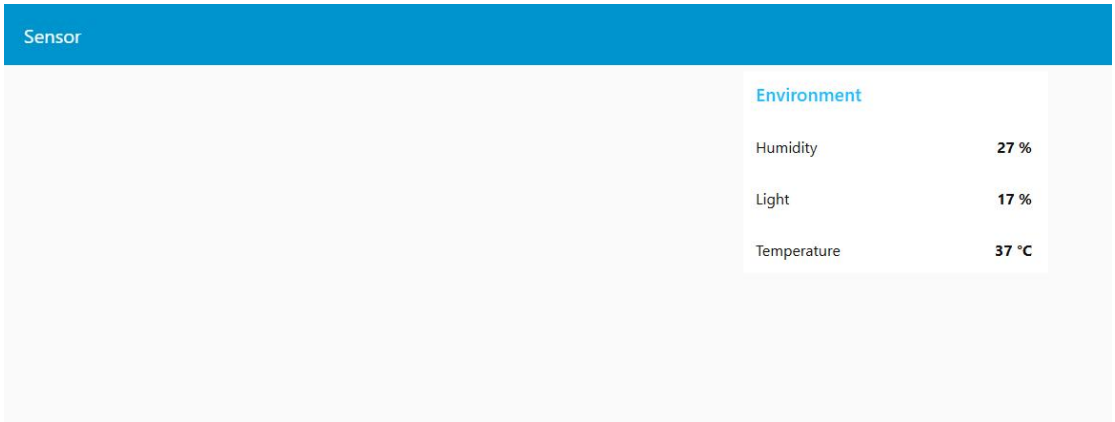
JSON：把 msg.payload 从字符串解析为对象

Debug：显示解析结果（右侧调试窗口）

HTTP Response：给 ESP8266 返回 200 OK，避免对端长时间等待

现在 Debug 已经能显示对象：temp/humi/light，说明 Node-RED 的接收与解析链路已经打通。





5. 系统联调与测试流程

5.1 阶段 1：传感器采集验证（串口输出）

目标：先确保 STM32 采集数据正确，再谈联网。

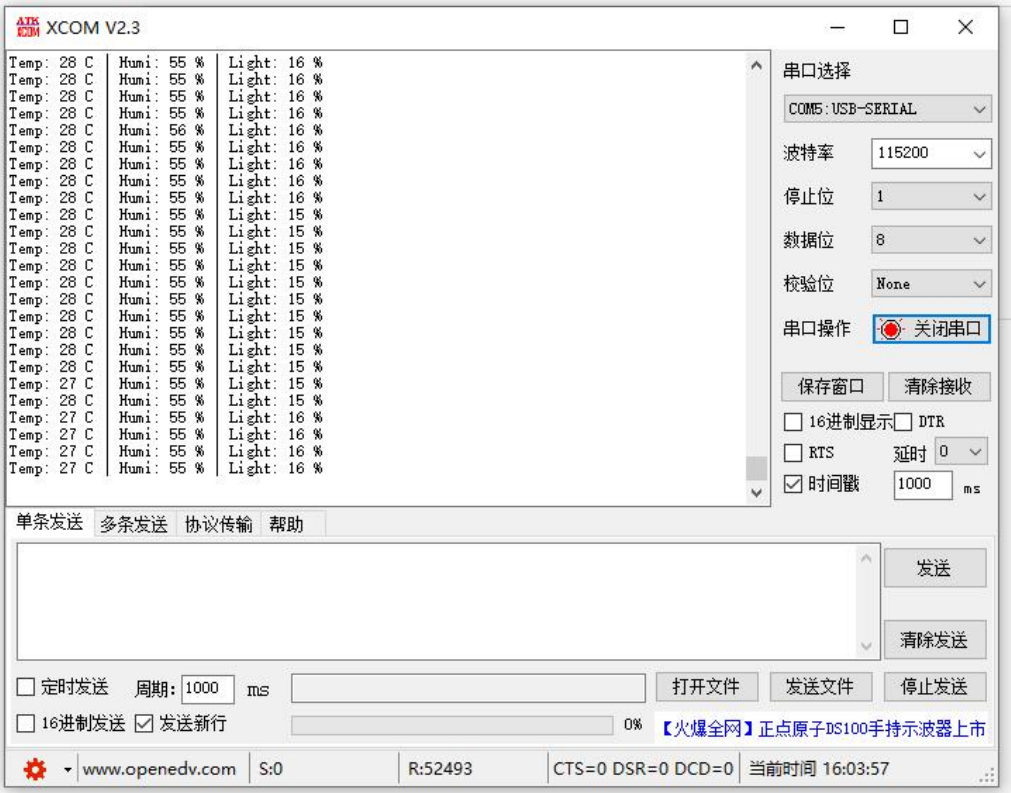
DHT11：初始化成功后定期读取 temp/humi

光敏：ADC 多次采样取平均并映射为 0–100%

串口打印：Temp=%d,Humi=%d,Light=%d\r\n

结果：在串口助手中已稳定看到温湿度与光照输出，证明采集层 OK。

理与封装，并通过无线网络上传至后端应用平台。



5.2 阶段 2：引入 ESP8266 后出现串口输出消失

现象：插上 ESP8266 后，串口助手不再显示 printf() 输出；拔掉后仍异常。

定位结论：USART1（PA9/PA10）被 ESP8266 占用并与板载串口/调试链路发生冲突，导致：

fputc() 写 USART1 的输出通道不可用

PC 串口调试链路断开或被干扰

处理原则：明确“哪一路串口给谁用”，将 ESP8266 移至独立 UART，或放弃 USART1 调试并改用 USART2。本质就是做串口资源重构。

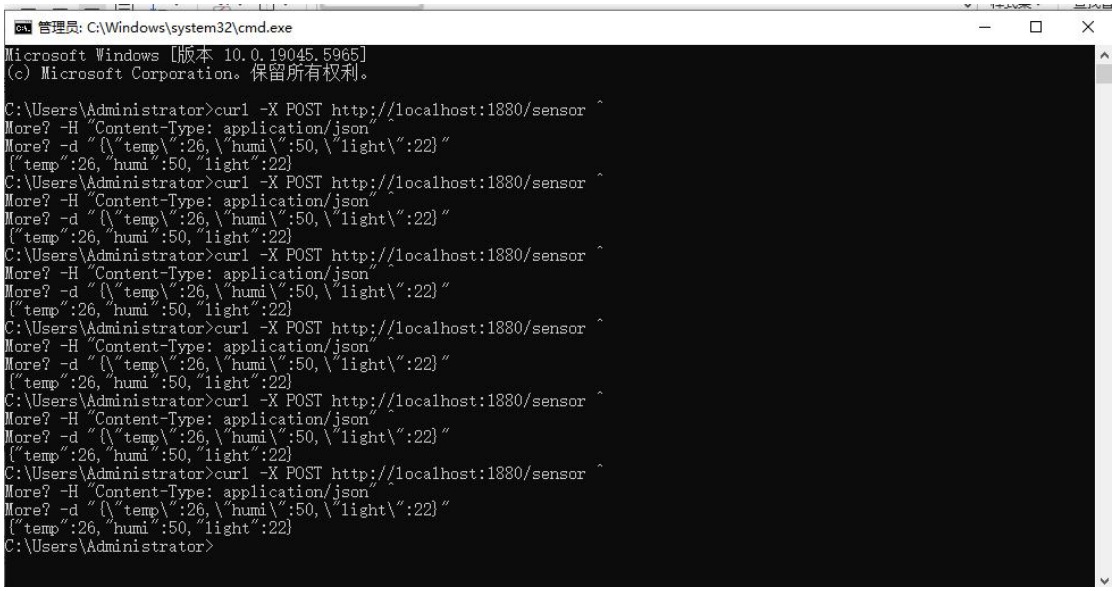
5.3 阶段 3：Node-RED 接收端搭建与 Debug 验证

目标：先让 Node-RED 能接到任何合法请求，再优化 STM32 上报。

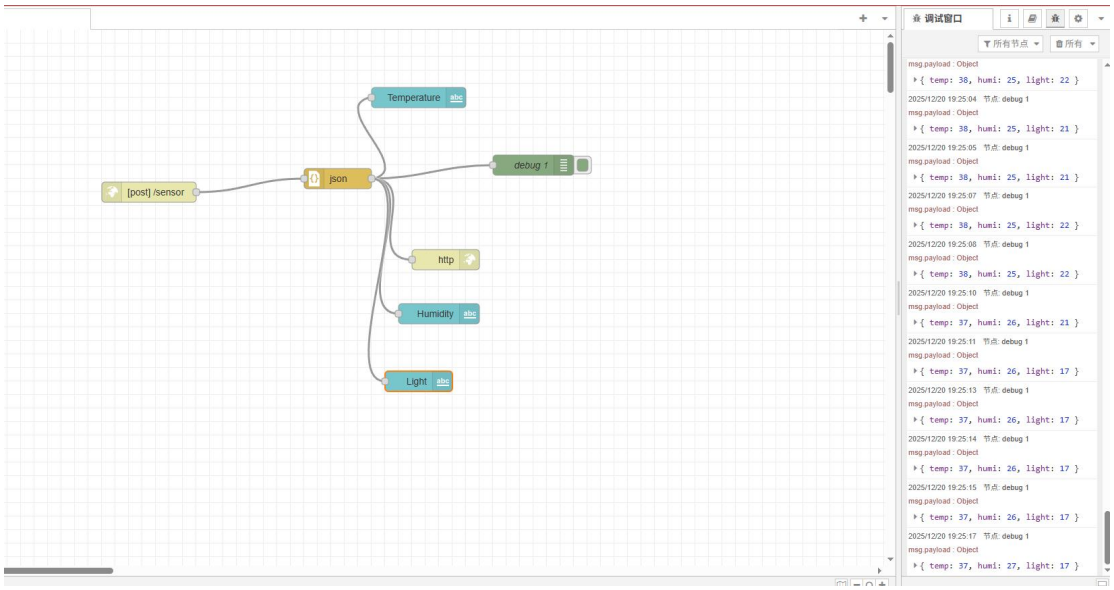
在 Node-RED 建立 /sensor 接口

Debug 节点查看 msg.payload

通过 cmd 发送指令，先模拟发送 JSON，确认流程正确



结果：你已能在 Debug 中看到结构化字段，接收端 OK。



5.4 阶段 4：网络环境问题（校园网不可直连）与解决

现象：使用校园 Wi-Fi（如 1x.fjnu.edu.cn）时 ESP8266 很可能无法真正联网成功。

原因：校园网常见“Portal/网页认证/802.1X/二次认证”等机制，ESP8266 这种纯 AT 连接通常无法完成网页认证流程，导致看似连接上 SSID，但实际不能访问你的 PC 服务。

解决：改用手机热点（WPA2-Personal、2.4GHz），ESP8266 可直接连接并获得可路由 IP。

结果：换热点后，Node-RED 成功收到数据，上报闭环完成。

5.5 阶段 5：端到端稳定性验证

采集周期稳定（0.2s 一次）

Node-RED Debug 连续收到多次数据（带时间戳）

断电重启后可自动恢复（ESP 上电延时 + 自动重连）

轻度丢包/失败可重试（CIPSTART/CIPSEND 返回异常则重连）

6. 结果展示与分析

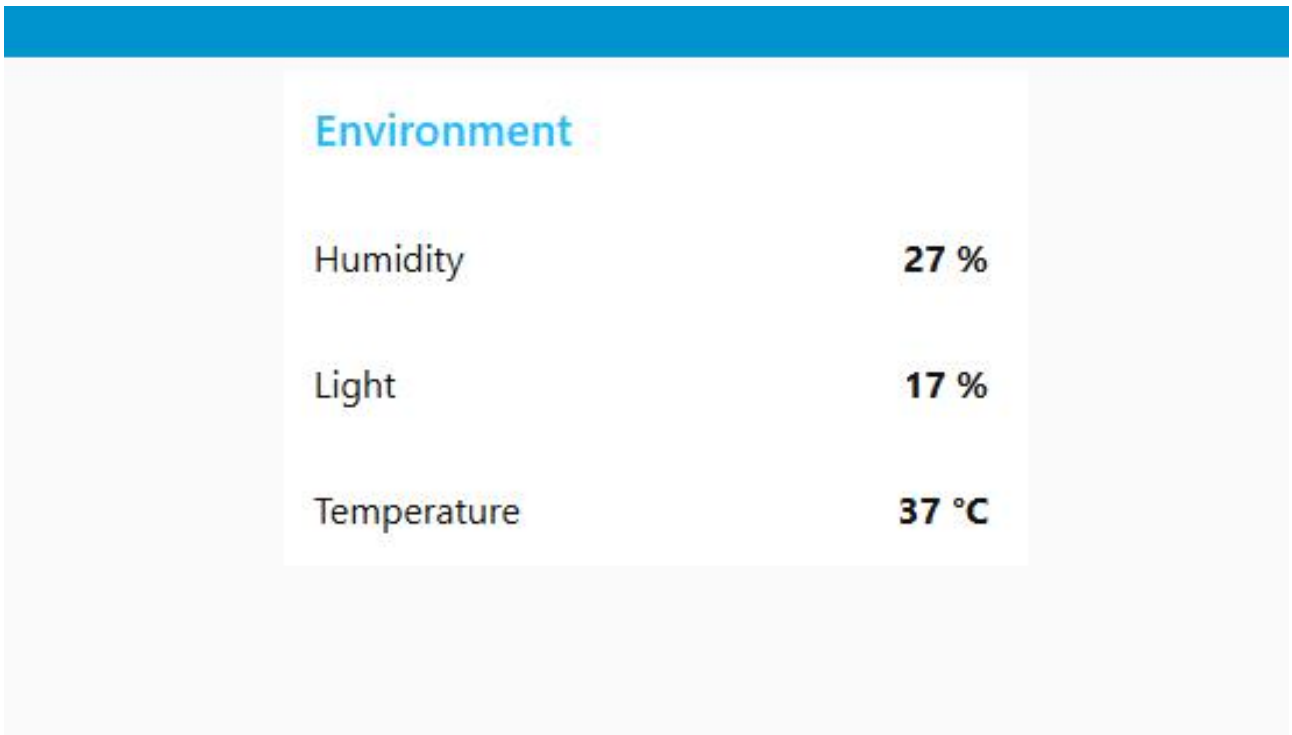
6.1 Node-RED Debug 结果

msg.payload 从字符串 JSON 成功解析为对象

字段包括 temp、humi、light

多次上报均可在调试窗口看到时间戳与数值变化

这已经满足“前后端交互、数据上传、平台可视化验证”的课程实操要求。



msg.payload : Object

▸ { temp: 38, humi: 25, light: 22 }

2025/12/20 19:25:04 节点: debug 1

msg.payload : Object

▸ { temp: 38, humi: 25, light: 21 }

2025/12/20 19:25:05 节点: debug 1

msg.payload : Object

▸ { temp: 38, humi: 25, light: 21 }

2025/12/20 19:25:07 节点: debug 1

msg.payload : Object

▸ { temp: 38, humi: 25, light: 22 }

2025/12/20 19:25:08 节点: debug 1

msg.payload : Object

▸ { temp: 38, humi: 25, light: 22 }

2025/12/20 19:25:10 节点: debug 1

msg.payload : Object

▸ { temp: 37, humi: 26, light: 21 }

2025/12/20 19:25:11 节点: debug 1

msg.payload : Object

▸ { temp: 37, humi: 26, light: 17 }

2025/12/20 19:25:13 节点: debug 1

msg.payload : Object

▸ { temp: 37, humi: 26, light: 17 }

2025/12/20 19:25:14 节点: debug 1

msg.payload : Object

▸ { temp: 37, humi: 26, light: 17 }

2025/12/20 19:25:15 节点: debug 1

msg.payload : Object

▸ { temp: 37, humi: 26, light: 17 }

2025/12/20 19:25:17 节点: debug 1

msg.payload : Object

▸ { temp: 37, humi: 27, light: 17 }

在此便可对收集到前端传感器数据的存储和分析

7. 关键问题复盘与工程化改进建议

7.1 串口冲突（USART1 被占用）

问题本质：硬件资源规划不足。

改进建议：设计之初先画出“引脚资源表”：传感器、调试串口、Wi-Fi 串口各占用哪些 IO

printf 重定向不写死 USART1，而是用宏 DEBUG_USARTx 可切换

ESP8266 与 PC 调试口必须物理隔离或通过跳帽切换

7.2 网络环境适配

校园网往往不可用，选择手机热点作为“可控网络环境”

给出 PC 的热点 IP（如 172.20.10.2）并固定 Node-RED 监听端口。

8. 结论

本项目完成了基于 STM32F103 的双传感器数据采集，并通过 ESP8266（AT 模式）将数据上传至 PC 端 Node-RED 平台，实现了从“采集—联网—上传—解析—展示”的完整闭环。系统实现过程中解决了 USART1 与 ESP8266 的资源冲突问题，并针对校园网认证环境不可用的限制，采用手机热点构建可路由网络，保证了端到端链路稳定可演示。整体方案结构清晰、实现成本低、可扩展性强，满足课程对物联网系统“前后端交互与数据处理流程”的要求。